JSON-RPC 서버 분석

1. JSON-RPC 서버 개요

JSON-RPC 서버는 Ethereum 호환 API를 제공하여 기존 Ethereum 도구들이 MiniEVM과 상호작용할 수 있도록 합니다.

1.1 지원하는 네임스페이스

- **cosmos**: Cosmos ↔ EVM 트랜잭션 해시 변환
- **web3**: Web3 유틸리티 함수들
- **net**: 네트워크 정보
- **eth**: 핵심 Ethereum API
- **debug**: 디버깅 및 트레이싱
- **txpool**: 트랜잭션 풀 관리
- **personal**: 계정 관리 (일부 미지원)

2. 지원되는 API 목록

2.1 Cosmos 네임스페이스

```
✅ cosmos_cosmosTxHashByTxHash - EVM → Cosmos 트랜잭션 해시 변환
✅ cosmos_txHashByCosmosTxHash - Cosmos → EVM 트랜잭션 해시 변환
```

2.2 Web3 네임스페이스

- ✅ web3_clientVersion - 클라이언트 버전 정보
- ✅ web3_sha3 - Keccak-256 해시 계산

2.3 Net 네임스페이스

- ✅ net_version - 네트워크 ID 반환
- ✅ net_peerCount - 연결된 피어 수
- ✅ net_listening - 네트워크 리스닝 상태

2.4 Eth 네임스페이스 (핵심 API)

네트워크 정보

- ✅ eth_syncing - 동기화 상태
- ✅ eth_chainId - 체인 ID
- ✅ eth_gasPrice - 현재 가스 가격
- ✅ eth_maxPriorityFeePerGas - 최대 우선순위 수수료
- ✅ eth_feeHistory - 수수료 히스토리
- ✅ eth_blockNumber - 최신 블록 번호

계정 관리

- ✅ eth_accounts - 소유 계정 목록
- ✅ eth_getBalance - 계정 잔액 조회

- ✅ eth_getTransactionCount - 계정 논스 조회

상태 조회

- ✅ eth_getStorageAt - 스토리지 값 조회
- ✅ eth_getCode - 컨트랙트 코드 조회

트랜잭션 처리

- ✅ eth_sendRawTransaction - 서명된 트랜잭션 전송
- ✅ eth_call - 상태 변경 없는 호출
- ✅ eth_estimateGas - 가스 사용량 추정

블록 정보

- ✅ eth_getBlockByHash - 해시로 블록 조회
- ✅ eth_getBlockByNumber - 번호로 블록 조회
- ✅ eth_getBlockReceipts - 블록의 모든 영수증
- ✅ eth_getTransactionCountByHash - 블록의 트랜잭션 수 (해시)
- ✅ eth_getBlockTransactionCountByNumber - 블록의 트랜잭션 수 (번호)

트랜잭션 정보

- ✅ eth_getTransactionByHash - 해시로 트랜잭션 조회
- ✅ eth_getTransactionByBlockHashAndIndex - 블록 해시와 인덱스로 조회

- ✅ eth_getTransactionByBlockNumberAndIndex - 블록 번호와 인덱스로 조회
- ✅ eth_getRawTransactionByHash - 원시 트랜잭션 데이터
- ✅ eth_getRawTransactionByBlockHashAndIndex - 원시 트랜잭션 (해시+인덱스)
- ✅ eth_getRawTransactionByBlockNumberAndIndex - 원시 트랜잭션 (번호+인덱스)
- ✅ eth_getTransactionReceipt - 트랜잭션 영수증

로그 및 필터

- ✅ eth_newFilter - 로그 필터 생성
- ✅ eth_newBlockFilter - 블록 필터 생성
- ✅ eth_newPendingTransactionFilter - 대기 트랜잭션 필터 생성
- ✅ eth_uninstallFilter - 필터 제거
- ✅ eth_getFilterChanges - 필터 변경사항 조회
- ✅ eth_getFilterLogs - 필터 로그 조회
- ✅ eth_getLogs - 로그 조회

2.5 Debug 네임스페이스

- ✅ debug_traceTransaction - 트랜잭션 트레이싱
- ✅ debug_traceBlockByNumber - 블록 트레이싱 (번호)
- ✅ debug_traceBlockByHash - 블록 트레이싱 (해시)
- ✅ debug_storageRangeAt - 스토리지 범위 조회

2.6 Txpool 네임스페이스

```
✅ txpool_content - 모든 대기/큐 트랜잭션
✅ txpool_inspect - 트랜잭션 풀 요약
✅ txpool_contentFrom - 특정 주소의 트랜잭션
✅ txpool_status - 트랜잭션 풀 상태
```

2.7 미지원 API

```
🚫 eth_coinbase - 코인베이스 주소
🚫 eth_mining - 마이닝 상태
🚫 eth_hashrate - 해시레이트
🚫 eth_sign - 메시지 서명
🚫 eth_sendTransaction - 트랜잭션 생성 및 전송
🚫 personal_* - 개인 계정 관리
🚫 eth_getProof - 머클 증명
```

3. JSON-RPC 서버 아키텍처

3.1 디렉토리 구조

```
jsonrpc/
├── backend/ # 백엔드 구현
├── config/ # 설정 관리
```

```
├── namespaces/ # 네임스페이스별 API 구현
├── types/ # JSON-RPC 타입 정의
├── jsonrpc.go # 메인 서버
├── logger.go # 로깅
└── README.md # API 문서
```

3.2 서버 구조

```go
type JSONRPCServer struct {
logger log.Logger
config config.Config
backend backend.Backend
httpServer *http.Server
// 네임스페이스 핸들러들
ethAPI *eth.PublicAPI
web3API *web3.PublicAPI
netAPI *net.PublicAPI
debugAPI *debug.PublicAPI
txpoolAPI *txpool.PublicAPI
cosmosAPI *cosmos.PublicAPI
}
```

4. 백엔드 시스템

4.1 Backend 인터페이스

```
type Backend interface {

// 블록체인 상태

ChainID() *big.Int

CurrentBlock() *types.Header

SuggestGasTipCap(ctx context.Context) (*big.Int, error)

// 계정 관리

GetBalance(address common.Address, blockNumber rpc.BlockNumber) (*big.Int, error

GetCode(address common.Address, blockNumber rpc.BlockNumber) ([]byte, error)

GetNonce(address common.Address, blockNumber rpc.BlockNumber) (uint64, error)

// 트랜잭션 처리

SendRawTransaction(ctx context.Context, tx *types.Transaction) error

EstimateGas(ctx context.Context, args TransactionArgs, blockNrOrHash *rpc.BlockN

// 블록 및 트랜잭션 조회

GetBlockByNumber(blockNumber rpc.BlockNumber, fullTx  bool) (map[string]interfac

GetTransactionByHash(hash common.Hash) (*RPCTransaction, error)

GetTransactionReceipt(hash common.Hash) (map[string]interface{}, error)

// 로그 및 필터

GetLogs(ctx context.Context, crit FilterCriteria) ([]*types.Log, error)

NewFilter(crit FilterCriteria) (rpc.ID, error)
```

```
GetFilterChanges(id rpc.ID) (interface{}, error)

}
```

4.2 백엔드 구현체

백엔드는 EVM 인덱서와 Cosmos SDK를 연결하는 어댑터 역할을 합니다:

```
type BackendImpl struct {

app *app.MinitiaApp

indexer indexer.EVMIndexer

clientCtx client.Context

logger log.Logger

// 필터 관리

filterSystem *FilterSystem

}
```

5. 네임스페이스별 구현

5.1 Eth 네임스페이스

가장 복잡하고 중요한 네임스페이스로, Ethereum의 핵심 기능을 제공합니다.

주요 메서드 구현 예시:

```
// eth_getBalance

func (api *PublicAPI) GetBalance(address common.Address, blockNrOrHash rpc.Block
```

```
state, _, err := api.backend.StateAndHeaderByNumberOrHash(ctx, blockNrOrHash)
if  err != nil {
return  nil, err
}
balance := state.GetBalance(address)
return (*hexutil.Big)(balance), nil
}


// eth_sendRawTransaction
func (api *PublicAPI) SendRawTransaction(ctx context.Context, input hexutil.Byte
tx := new(types.Transaction)
if  err := tx.UnmarshalBinary(input);  err != nil {
return common.Hash{}, err
}
return  api.backend.SendRawTransaction(ctx, tx)
}


// eth_call
func (api *PublicAPI) Call(ctx context.Context, args TransactionArgs, blockNrOrH
result, err := api.backend.DoCall(ctx, args, blockNrOrHash, overrides, api.backe
if  err != nil {
return  nil, err
}
```

```
return  result.Return(), nil
}
```

5.2 Debug 네임스페이스

트랜잭션 실행 과정을 상세히 추적할 수 있는 디버깅 API를 제공합니다.

```
// debug_traceTransaction
func (api *DebugAPI) TraceTransaction(ctx context.Context, hash common.Hash, con
tx, blockHash, blockNumber, index, err := api.backend.GetTransaction(ctx, hash)
if  err != nil {
return  nil, err
}
return  api.backend.TraceTransaction(ctx, tx, blockHash, blockNumber, index, con
}
```

5.3 Cosmos 네임스페이스

Cosmos와 EVM 간의 트랜잭션 해시 매핑을 제공하는 MiniEVM 고유 기능입니다.

```
// cosmos_cosmosTxHashByTxHash
func (api *CosmosAPI) CosmosTxHashByTxHash(ctx context.Context, hash common.Hash
cosmosTxHash, err := api.backend.CosmosTxHashByTxHash(ctx, hash)
if  err != nil {
```

```
return  nil, err
}
return  hexutil.Bytes(cosmosTxHash), nil
}


// cosmos_txHashByCosmosTxHash
func (api *CosmosAPI) TxHashByCosmosTxHash(ctx context.Context, hash hexutil.Byt
return  api.backend.TxHashByCosmosTxHash(ctx, hash)
}
```

6. 필터 시스템

6.1 필터 타입

- **로그 필터**: 특정 조건의 이벤트 로그 필터링
- **블록 필터**: 새로운 블록 알림
- **대기 트랜잭션 필터**: 메모리풀의 새 트랜잭션 알림

6.2 필터 관리

```
type FilterSystem struct {
backend Backend
// 필터 저장소
filters  map[rpc.ID]*Filter
```

```
// 구독 관리
blockSub Subscription
logsSub Subscription
pendingSub Subscription
}
```

6.3 필터 생성 및 관리

```
// eth_newFilter
func (api *PublicAPI) NewFilter(crit FilterCriteria) (rpc.ID, error) {
filter := &Filter{
typ: LogsSubscription,
criteria: crit,
logs: make(chan []*types.Log),
}
id := api.filterSystem.NewFilter(filter)
return  id, nil
}

// eth_getFilterChanges
func (api *PublicAPI) GetFilterChanges(id rpc.ID) (interface{}, error) {
return  api.filterSystem.GetFilterChanges(id)
}
```

7. 트랜잭션 처리 플로우

7.1 eth_sendRawTransaction 처리 과정

1. **트랜잭션 파싱**: 원시 바이트를 Ethereum 트랜잭션으로 파싱
2. **서명 검증**: 트랜잭션 서명 유효성 검사
3. **Cosmos 트랜잭션 변환**: EVM 트랜잭션을 Cosmos 트랜잭션으로 래핑
4. **메모리풀 추가**: 트랜잭션을 메모리풀에 추가
5. **인덱서 알림**: 대기 트랜잭션으로 인덱서에 알림

7.2 eth_call 처리 과정

1. **상태 스냅샷**: 지정된 블록의 상태 스냅샷 생성
2. **EVM 실행**: 상태 변경 없이 EVM에서 실행
3. **결과 반환**: 실행 결과 및 가스 사용량 반환

8. 성능 최적화

8.1 인덱서 활용

- 모든 쿼리는 인덱서를 통해 처리
- 실시간 데이터 캐싱
- 블룸 필터를 통한 로그 검색 최적화

8.2 캐싱 전략

- 블록 헤더 캐싱
- 트랜잭션 영수증 캐싱

- 상태 스냅샷 캐싱

8.3 배치 처리

- 여러 요청을 배치로 처리
- 데이터베이스 쿼리 최적화
- 메모리 사용량 최적화

9. 에러 처리 및 호환성

9.1 Ethereum 호환 에러 코드

```
const (
ErrCodeParseError = -32700
ErrCodeInvalidRequest = -32600
ErrCodeMethodNotFound = -32601
ErrCodeInvalidParams = -32602
ErrCodeInternalError = -32603
)
```

9.2 에러 응답 형식

```
{
"jsonrpc": "2.0",
"id": 1,
```

```
"error": {
"code": -32602,
"message": "Invalid params",
"data": "additional error information"
}
}
```

이러한 JSON-RPC 서버 구현을 통해 MiniEVM은 기존 Ethereum 생태계의 모든 도구들과 완벽하게 호환되면서도 Cosmos 생태계의 고유 기능들을 활용할 수 있습니다.